ECE 385

Final Project

KeySync

Fall 2025

Apoorva Sharma
NetID: as204

Samridhi Verma
NetID: sv49

Introduction

This project implements a real-time rhythm game on an FPGA using SystemVerilog. The game features four gameplay lanes with falling notes that respond to keyboard input, along with real-time score, lives, and game state tracking. Video output is generated using a VGA interface.

The system integrates multiple hardware subsystems, including a finite state machine for game control, lane-based graphics rendering, hit/miss detection logic, and an audio pipeline using ROM-based sample playback and PWM output.

Figure 1 shows the block diagram for our project implementation.

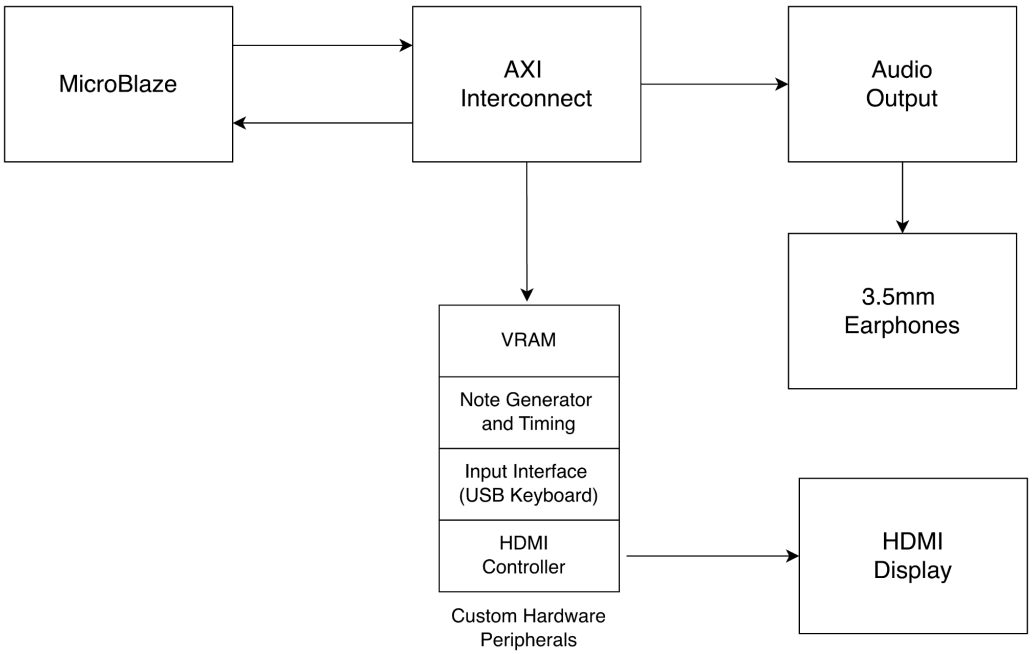

Figure 1. Block Diagram

Audio

Audio System Implementation

The game includes an audio playback system that outputs music during gameplay. The audio system consists of ROM-based sample storage, controlled sample playback, and clocked pulse-width modulation (PWM).

Audio Sample Storage

We stored the audio samples in on-chip block RAM (BRAM), configured the BRAM as a read-only memory (ROM), and initialized it at synthesis time from the COE file. The audio samples were

recorded at a sampling rate of 44.1 kHz with an 8-bit resolution to consider the limited storage capacity of the BRAM.

To prepare the audio data, the original MP3 file was first converted to a mono, signed 8-bit PCM RAW format using Audacity. A Python script was then used to convert the RAW file into a COE file, which was subsequently loaded into BRAM.

PWM Clock Generation

To generate an audible signal suitable for analog output, a dedicated PWM clock is derived from the 100 MHz system clock using a clock divider. The pwm_clk_gen module produces a lower-frequency clock enable signal by dividing the system clock by a fixed factor.

This clock division ensures that the PWM carrier frequency falls within a range that the onboard low-pass filter and external audio hardware can effectively filter. Separating PWM timing from the main system clock improves audio quality and prevents high-frequency artifacts.

Pulse-Width Modulation (PWM) Audio Output

The pwm_audio module converts digital audio samples into a PWM signal suitable for output to an audio jack. An internal counter increments at the PWM clock rate, and the output signal is asserted when the counter value is less than the current audio sample value.

This comparison-based PWM approach encodes audio amplitude as duty cycle, allowing the average output voltage to represent the audio waveform after low-pass filtering.

Graphics

Four-Lane Gameplay

The game display is divided into four vertical lanes. Each lane has a fixed width and horizontal offset, calculated relative to the VGA horizontal resolution.

During rendering, the current pixel's x-coordinate is compared against the boundaries of each lane to determine which lane the pixel belongs to. This lane index is then used by other modules, including the tile renderer and hit detection logic, to associate falling notes with the correct lane. By maintaining a consistent mapping between lane index and keyboard input, the visual layout remains synchronized with gameplay controls.

This lane-based structure simplifies both rendering and game logic by providing a clean space for note movement, collision detection, and scoring.

Gradient Background

To enhance visual clarity and aesthetic quality, the background employs a color gradient instead of a solid color. The gradient is computed dynamically based on the pixel's y-coordinate, varying from a darker color at the top of the screen and a lighter color at the bottom.

The gradient is implemented using simple arithmetic on the RGB channels, avoiding the use of large lookup tables or memory blocks. This ensures minimal FPGA resource usage while still achieving a smooth visual transition. The background rendering module outputs RGB values only when no foreground elements are active for the current pixel.

The gradient background helps distinguish falling notes from the playfield and provides depth to the game's visual design without impacting performance.

HUD Rendering

The Heads-Up Display (HUD) presents gameplay information such as score and remaining lives. The HUD occupies a reserved region of the screen and is rendered independently from the gameplay lanes to avoid overlap with falling notes.

HUD elements are drawn using pixel-level bounding boxes that define where each element appears on screen. Each HUD component is enabled conditionally based on the current game state. The rendering logic prioritizes HUD pixels over background pixels ensuring clear visibility without interfering with gameplay.

By separating HUD rendering into its own logic block, the design remains modular and allows additional UI elements to be added without modifying the core game renderer.

Text Rendering

Text rendering is implemented using a character ROM that stores bitmap representations of alphanumeric characters and symbols. Each character is defined as a small pixel grid, allowing text such as "SCORE," numeric values, and game state messages to be displayed on screen.

For each pixel, the renderer determines whether it lies within a text region. If it does, the corresponding character index and pixel offset within that character are used to read from the font ROM. This approach enables dynamic text updates, such as changing scores and combo counts, without requiring pre-rendered images.

This method is memory-efficient and well-suited for FPGA implementation, as it avoids frame buffers while still allowing flexible and readable on-screen text.

Real-Time Falling Note Display

Falling notes are rendered in real time as rectangular tiles that descend vertically within one of four gameplay lanes. Each note is defined by a lane index, a vertical position, and an active flag. The horizontal position is computed directly from the lane index, ensuring visual alignment with the corresponding player input.

Notes are generated using a pseudo-random algorithm based on a linear feedback shift register (LFSR). The LFSR produces a sequence of pseudo-random values, which are mapped to lane indices to determine where new notes spawn. This approach provides sufficient randomness for gameplay while remaining hardware-efficient and fully synthesizable on the FPGA.

The vertical motion of notes is controlled by a speed parameter that increases gradually over time to raise game difficulty. On each game tick, the note's y-position is incremented by the current speed value. The speed is updated at fixed time intervals using a clock-driven counter, allowing difficulty to scale smoothly without abrupt changes.

During VGA rendering, the current pixel coordinates are compared against the bounding boxes of all active notes. If a match occurs, the note is drawn instead of the background. Notes are removed once they are successfully hit or fall past the bottom of the screen. This design ensures smooth animation, synchronized gameplay, and a clear progression in challenge.

Hit/Miss Detection

Hit and miss detection is tightly coupled with the visual representation of falling notes. A fixed "hit zone" is defined near the bottom of the screen. When a note enters this zone, player input is evaluated to determine whether the note was successfully hit.

If the correct key is pressed while a note is within the hit zone, the note is marked as hit and removed from the active note list. If the note passes through the zone without a valid input, it is marked as missed. Visual feedback is provided by updating the HUD.

This alignment between visual position and gameplay logic ensures intuitive and responsive user interaction.

Score and Lives Tracking

The game implements a real-time score and lives tracking system to provide continuous feedback on player performance. The score represents the number of successful note hits, while the lives counter reflects the number of remaining mistakes the player is allowed before the game ends.

Each time a falling note enters the hit zone, player input is evaluated. If the correct key corresponding to the note's lane is pressed within the allowed timing window, the note is marked as a successful hit. On a successful hit, the score register is incremented by 1 and the note is removed from active gameplay. If a note passes through the hit zone without a valid input, it is counted as a miss, and the player's lives counter is decremented.

Both score and lives are stored in synchronous registers and updated on the game clock. When the lives counter reaches zero, the game transitions to the Game Over state, freezing gameplay while preserving the final score for display.

Game States

The game operates under a finite state machine with states such as Start Screen, Gameplay, and Game Over. Each state controls both gameplay behavior and what is rendered on screen.

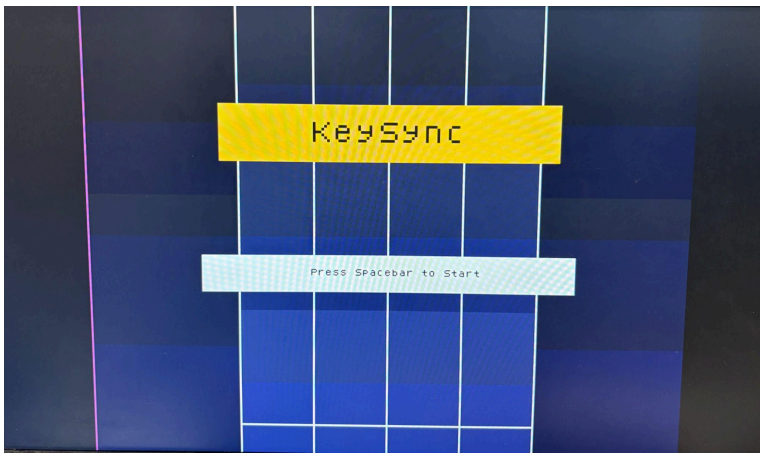

Figure 2. Game Start Screen

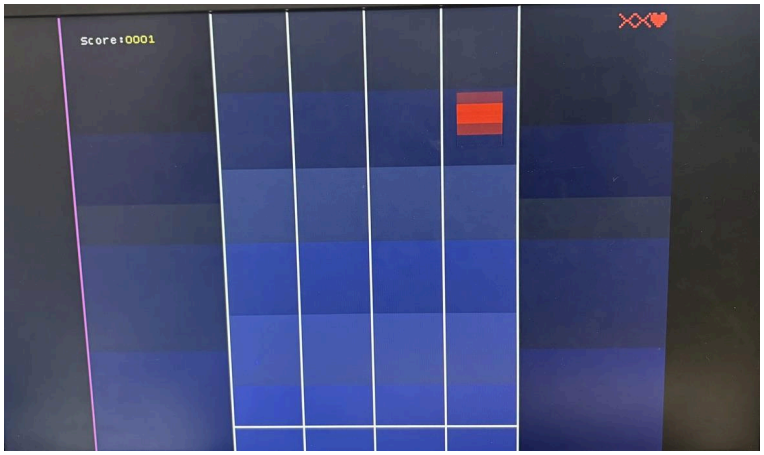

Figure 3. Gameplay Screen

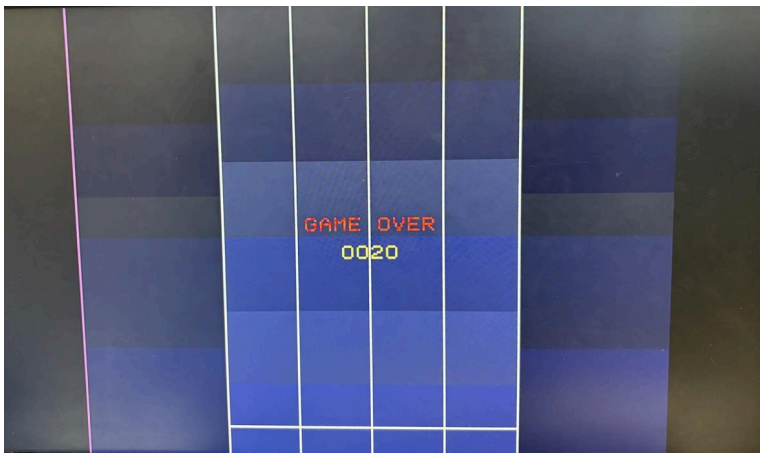

Figure 4. Game Over Screen

For example, the start screen (*Figure 1*) displays instructional text, the gameplay state (*Figure 2*) enables falling notes and scoring, and the game over state (*Figure 3*) freezes gameplay while displaying the final score. Rendering logic checks the current state before enabling certain visual elements, ensuring that only relevant graphics are shown. The user can go from the Game Over state back to the Game Start state by pressing the spacebar.

This state-driven design improves clarity, prevents unintended behavior, and makes the system easier to debug and extend.

Input and Output Interfaces

Player input is provided through a USB keyboard interface. Keycodes from the keyboard are decoded to detect lane presses during gameplay as well as spacebar presses used to transition between game states. This input is synchronized with the game clock to ensure accurate hit and miss detection aligned with the visual position of falling notes.

Audio output is delivered through the board's 3.5 mm audio jack, allowing standard earphones or speakers to be used.

Module Descriptions

Besides the modules already described in the Lab 6 report, this project introduces several additional modules to support extended functionality such as gameplay logic, enhanced graphics, and audio output.

Module: **`game_fsm.sv`**

Inputs: `clk, reset, space_pressed, [1:0] miss_count`

Outputs: `game_state_t state, restart_game`

Description**:** This module implements the finite state machine that controls the overall game flow. It transitions between the start screen, gameplay, and Game Over states based on player input and the number of missed notes.

Purpose**:** Manages high-level game control and ensures proper state transitions, including restarting gameplay and resetting score and lives.

Module**: `hud_renderer.sv`**

Inputs**:** `clk, reset, game_active, tile_hit, tile_missed, pixel_x, pixel_y, active, restart_game`

Outputs**:** `hud_on, [7:0] R, G, B, [15:0] score_out, [1:0] misses_out`

Description**:** This module tracks and renders the heads-up display (HUD), including the player's score and remaining lives. It updates score and miss counters based on gameplay events and converts numerical values into ASCII characters for text rendering.

Purpose**:** Provides real-time visual feedback to the player by displaying score and lives during gameplay.

Module**: `lane_background.sv`**

Inputs**:** `pixel_x, pixel_y, active`

Outputs**:** `[7:0] R, G, B`

Description**:** This module renders the game background, including a vertical color gradient, lane dividers, side regions, and a horizontal hit line. Pixel color is determined based on position and lane boundaries.

Purpose**:** Establishes the visual playfield structure and improves clarity by separating lanes and highlighting the hit region.

Module**: `rhythm_game.sv`**

Inputs**:** `clk_pix, reset, vsync, [31:0] keycode, drawX, drawY, active`

Outputs**:** `[7:0] R8, G8, B8`

Description**:** This is the top-level gameplay module that integrates all major subsystems, including input handling, tile generation and movement, hit and miss detection, HUD rendering, background rendering, and screen state selection.

Purpose**:** Serves as the main integration point for the rhythm game, coordinating graphics, gameplay logic, and state-based rendering.

Module: **`start_screen_renderer.sv`**

Inputs: `[9:0] pixel_x, [9:0] pixel_y, active`

Outputs: `draw_on, [7:0] R, G, B`

Description: This module renders the start screen, including the game title and a blinking prompt instructing the player to press the spacebar to begin. It uses frame-based timing to generate a blinking effect and multiple text renderers for title and prompt text.

Purpose: Displays an introductory screen and provides visual guidance before gameplay starts.

Module: **`game_over_renderer.sv`**

Inputs: `[9:0] pixel_x, [9:0] pixel_y, active, [15:0] final_score`

Outputs: `gameover_on, [7:0] R, G, B`

Description: This module renders the Game Over screen, including centered "GAME OVER" text and the player's final score using scaled text rendering.

Purpose: Displays end-of-game feedback once all lives are lost.

Module: **`text_renderer.sv`**

Inputs: `[9:0] pixel_x, [9:0] pixel_y, active, [9:0] text_x, [9:0] text_y, message`

Outputs: `text_on, [7:0] R, G, B`

Description: This module renders ASCII text on the VGA display by mapping characters to glyphs stored in a font ROM. It supports configurable text length and scaling, and determines whether each pixel belongs to an active glyph.

Purpose: Provides a reusable text-drawing primitive used for HUD elements, menus, and game state messages.

Module: **`tile_manager.sv`**

Inputs: `clk, reset, game_tick, [7:0] tile_speed, tile_hit`

Outputs: `tile_lane, tile_y, tile_active`

Description: This module manages the lifecycle of falling tiles, including spawning, vertical movement, and deactivation. Tile lanes are chosen using an LFSR-based pseudo-random generator, and tile positions are updated each game tick according to the current speed.

Purpose: Controls the creation and motion of falling notes during gameplay.

Module: **`tile_speed_controller.sv`**

Inputs: `clk, reset, game_tick`

Outputs: `[7:0] tile_speed`

Description: This module increases the falling speed of tiles over time by counting game ticks and incrementing a speed level at fixed intervals, up to a defined maximum.

Purpose: Implements difficulty scaling by gradually increasing gameplay speed.

Module: **`tile_renderer.sv`**

Inputs: `[9:0] pixel_x, [9:0] pixel_y, active, tile_lane, tile_y, tile_active`

Outputs: `tile_on, [7:0] R, G, B`

Description: This module renders falling tiles by checking whether the current pixel lies within the bounding box of any active tile and outputs the corresponding tile color.

Purpose: Visually displays falling notes in real time.

Module: **`audio_player.sv`**

Inputs: `clk_100mhz, reset_n, play, sample_en, rom_sample`

Outputs: `rom_addr, [7:0] audio_sample, sample_valid`

Description: This module controls audio playback by sequentially reading samples from ROM and outputting them at the audio sample rate. Signed samples are offset to unsigned format for PWM compatibility, and playback wraps automatically at the end of the sample buffer.

Purpose: Manages timing and control of digital audio sample playback.

Module: **`audio_rom.sv`**

Inputs: `clk, [16:0] address`

Outputs: `[15:0] q`

Description: This module implements a read-only memory initialized from a .COE file to store audio samples used for sound effects.

Purpose: Provides on-chip storage for preloaded audio data.

Module: **`pwm_audio.sv`**

Inputs: `clk_100mhz, pwm_ce, [7:0] sample`

Outputs: `pwm_out`

Description: This module converts digital audio samples into a pulse-width modulated signal by comparing each sample against a running counter.

Purpose: Generates an audio waveform suitable for analog output after low-pass filtering.

MicroBlaze System Block Diagram

Figure 5 shows the block diagram for the MicroBlaze SoC.

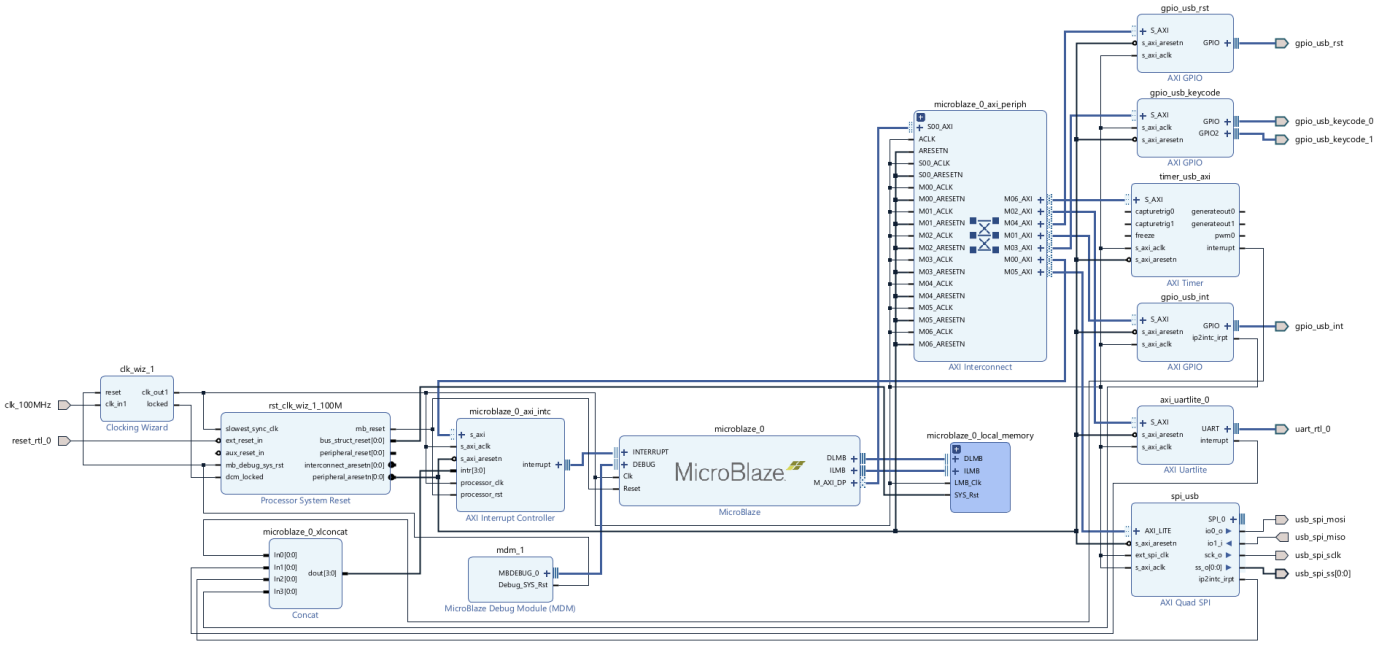

Figure 5. Block Diagram for MicroBlaze SoC

Top Level Block Diagram

Figure 6 shows the top-level diagram for our final project.

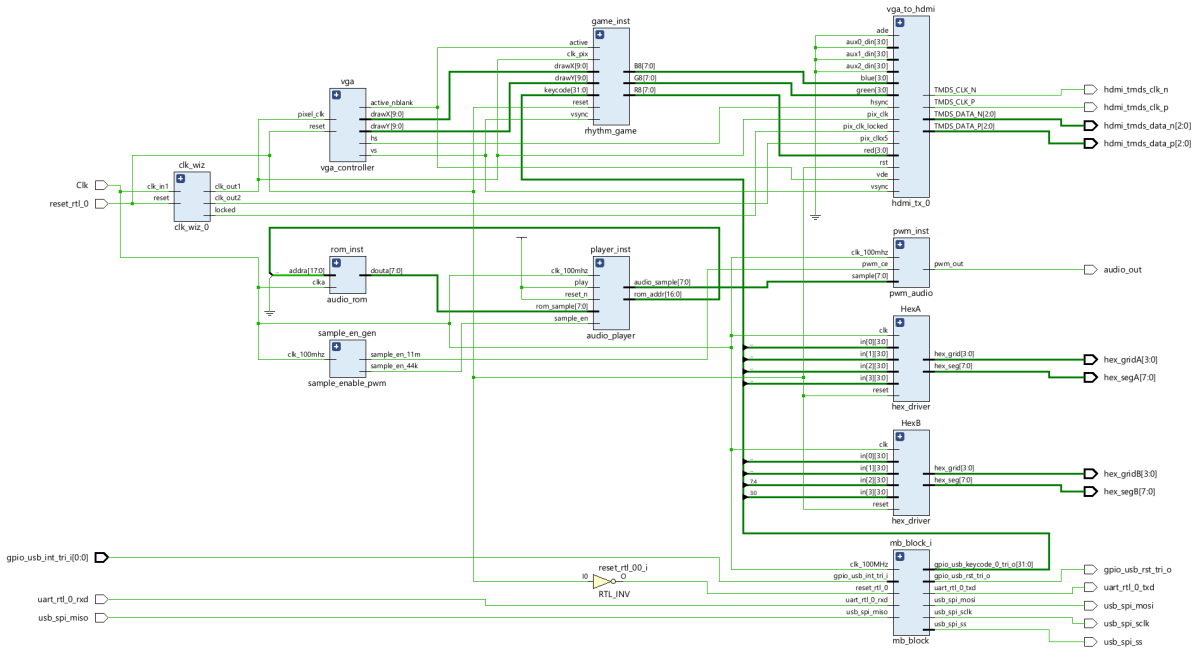

Figure 6. Top Level Diagram

Design Resources and Statistics

Table 1 shows the design and resource statistics for our final project.

Table 1. Design Resources and Statistics

LUT	4880
DSP	6
Memory (BRAM)	52
Flip-Flop	2849
Latches	0
Frequency	126.4 MHz
Static Power	0.078 W
Dynamic Power	0.389 W
Total Power	0.467 W

Conclusion

This project successfully integrates real-time graphics, input handling, and audio playback to create a fully functional FPGA-based rhythm game. The final design demonstrates how multiple hardware subsystems can operate together synchronously to support interactive gameplay.

One of the primary challenges encountered was implementing audio playback within limited BRAM resources while maintaining a 44.1 kHz sampling frequency necessary for acceptable audio quality. Balancing memory constraints with timing accuracy required careful sample sizing, clock division, and playback control. Additionally, synchronizing hit and miss detection with the visual position of falling notes proved challenging, as gameplay logic had to align precisely with rendered tile positions to ensure responsive input handling.

Despite these challenges, the final system achieves smooth visual animation, reliable audio output, and accurate gameplay feedback.